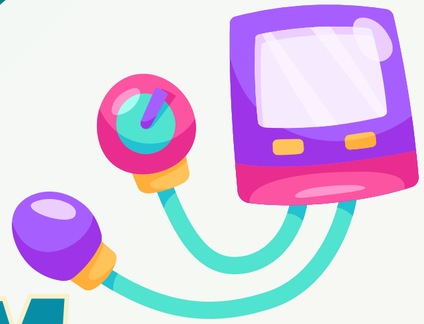
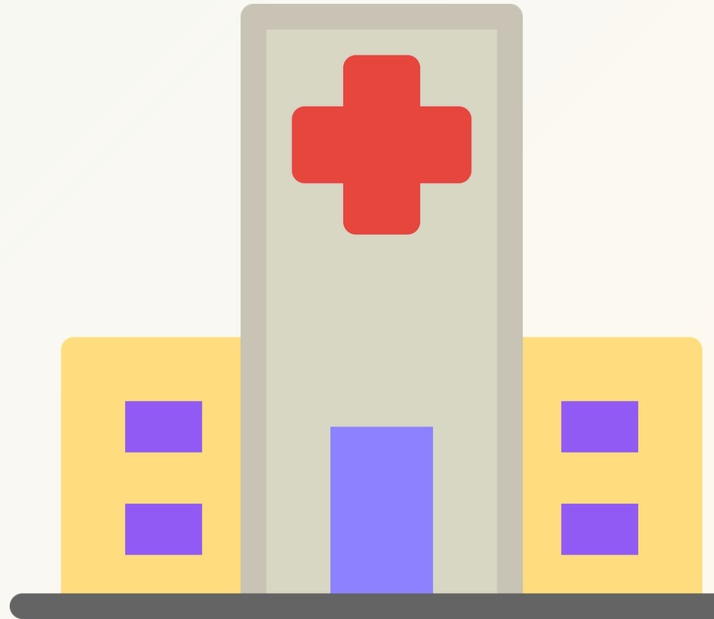
Database Systems
Semester Presentation

SMART HOSPITAL MANAGEMENT SYSTEM

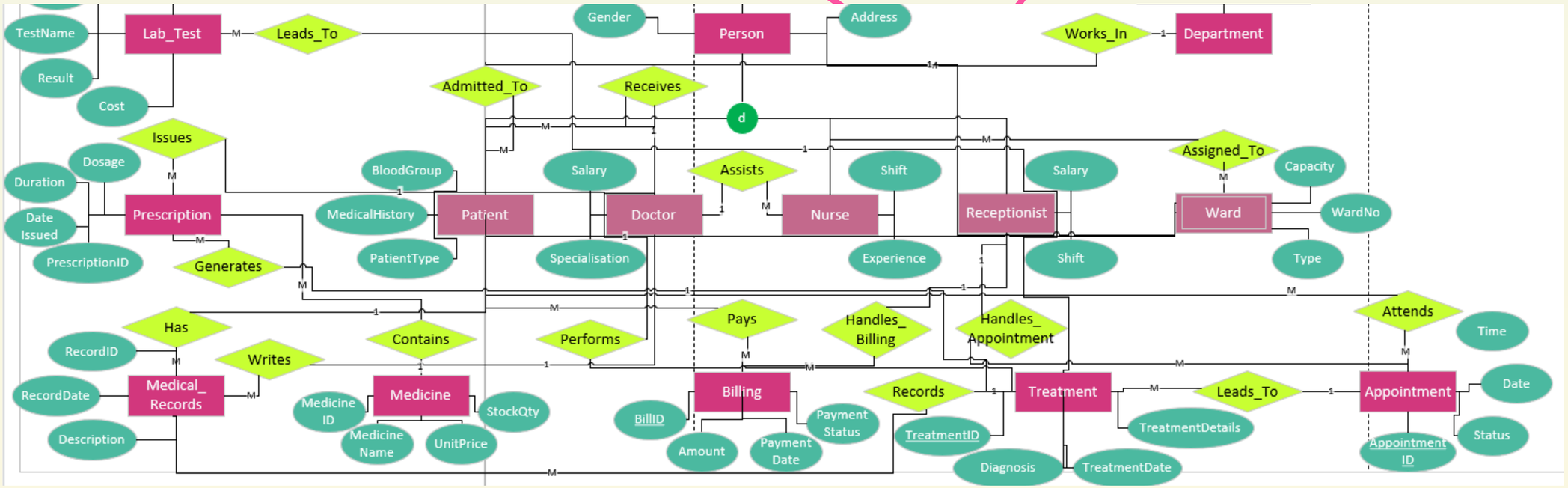


Presented By:

24F-0008 : Eesha Saqib
24F-0001 : Rameen Ahmad



EERD



- Shows all entities and relationships visually
- Uses specialization (Person → Patient/Doctor/Nurse/Receptionist)

- **Includes**:
- Primary Keys
- Foreign Keys
- Relationship constraints
- Person entity is generalized into multiple roles

Project overview

- A centralized database system for hospital management
- Integrates administrative, medical, and financial operations
- Eliminates manual record keeping
- Ensures accuracy, security, and efficiency

Core Modules

- Patient Management
- Doctor & Staff Management
- Appointments & Treatments
- Billing & Pharmacy

LIST OF TABLES

Basic Entities

- Person
- Patient
- Doctor
- Nurse
- Receptionist
- Department
- Ward

Medical Tables:

- Treatment
- Medical_Record
- Prescription
- Medicine
- Lab_Test

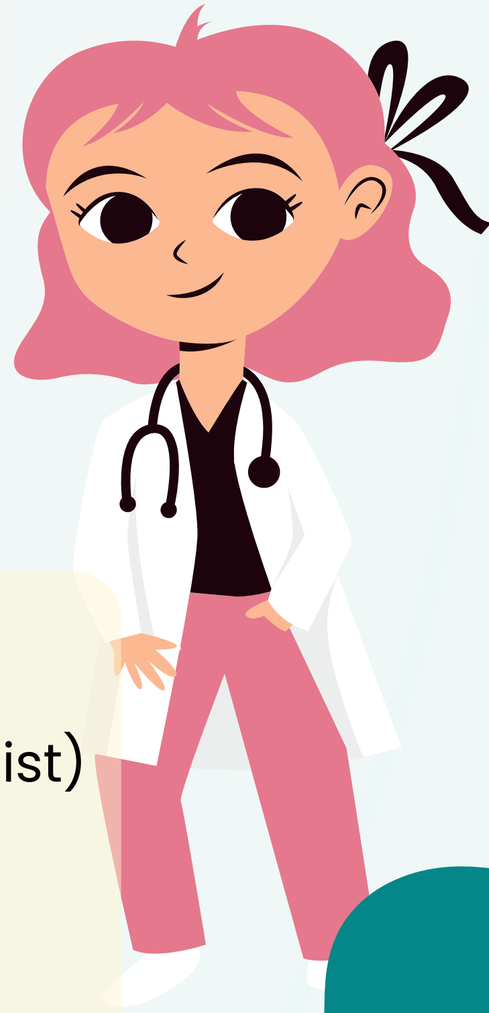
Operations & Billing

- Appointment
- Attends (Patient ↔ Appointment)
- Leads_To (Appointment → Treatment)
- Billing
- Pays (Patient ↔ Billing)
- Handles_Appointment
- Handles_Billing

Relational & Medical

- Works_In (Staff ↔ Department)
- Assigned_To (Nurse ↔ Ward)
- Admitted_To (Patient ↔ Ward)
- Receives (Doctor ↔ Patient)
- Assists (Nurse ↔ Doctor)

Enhanced ER Diagram



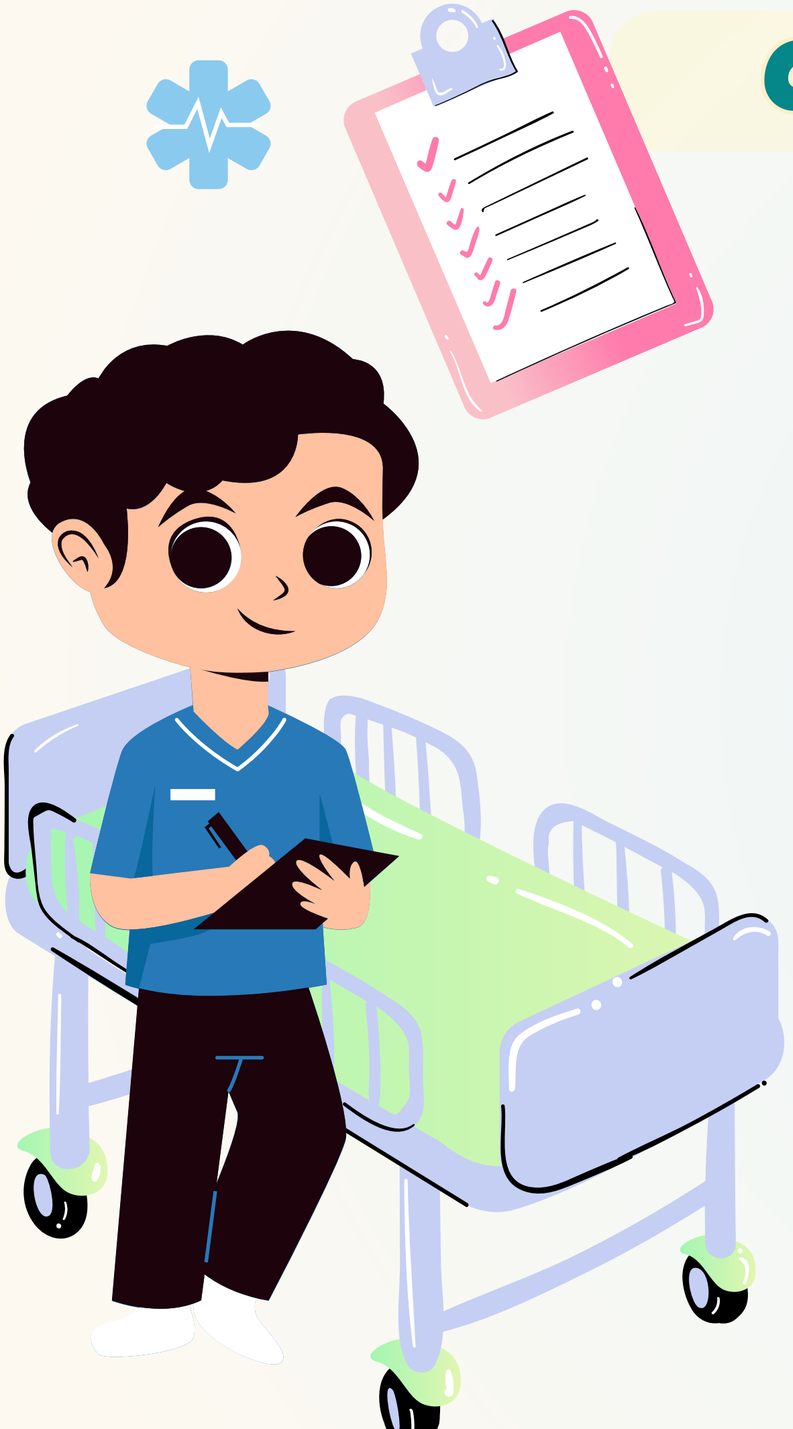
Shows all entities and relationships visually

Uses specialization (Person → Patient/Doctor/Nurse/Receptionist)

Includes:
Primary Keys
Foreign Keys
Relationship constraints

Person entity is generalized into multiple roles

Creating Views



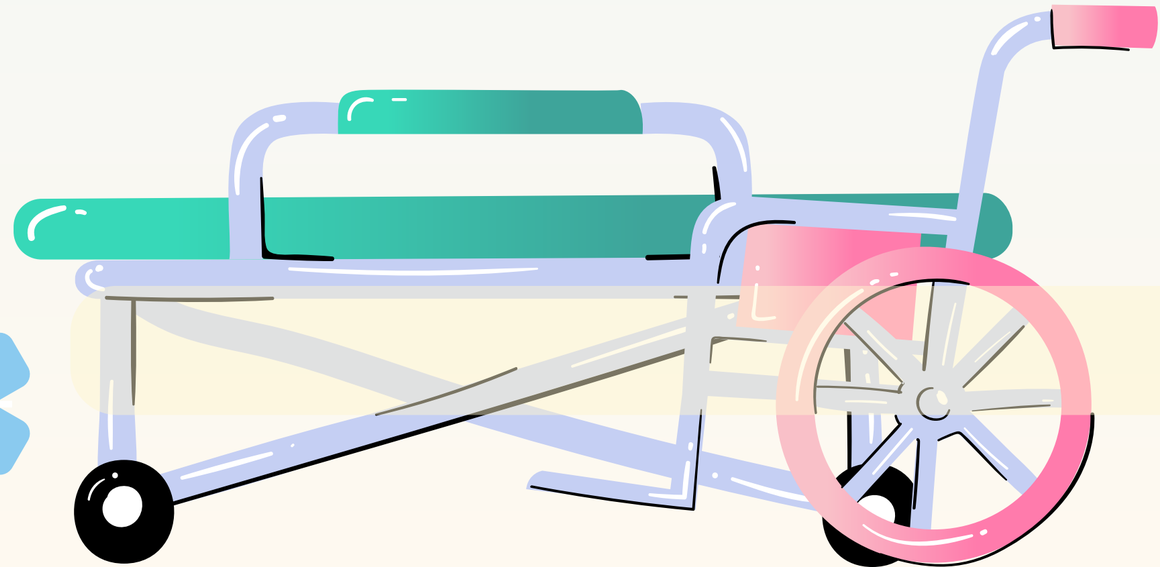
- In our system, views were designed to simplify complex queries and provide controlled data access.
- We created views such as Patient Directory, Doctor Profile, Appointment Booking, and Medical History.
- The main thought process was to hide complex joins and present user-friendly virtual tables.
- Views allow different roles (doctor, nurse, receptionist) to see only relevant data.
- For example, the Patient Directory view combines Person and Patient tables
- The Doctor Profile view integrates Person, Doctor, and Department (Works_In) tables.
- We also created dynamic views like Today's Appointments using system date filtering.
- Views improve security by restricting access to sensitive columns like salary.
- They reduce query complexity and improve maintainability of th
- Additionally, we used INSTEAD OF triggers on views to allow inse operations.

INDEXES

Indexes were implemented to improve query performance and speed up data retrieval.

- We created indexes on frequently used columns such as patientID, doctorID, treatmentID, and dates
- The thought process was to optimize queries involving joins, filtering, and searching.
- For example, indexing Medical_Record(patientID) speeds up patient history queries.
- Indexes on Billing(payment_date) help in financial reporting queries.
- We also indexed foreign keys to enhance join performance.
- Columns used in WHERE clauses and ORDER BY were prioritized.

- Indexing improves efficiency especially for large datasets.
- The balance between performance and storage was carefully maintained.
- Overall, indexes ensure faster execution of real-time hospital operations.



TRIGGERS

- Triggers were used to automate critical business rules in the database.
- They execute automatically on events like INSERT, UPDATE, or DELETE.
- Our thought process was to enforce data integrity at the database level.
- For example, a trigger reduces medicine stock when a prescription is added.
- Another trigger prevents appointments from being scheduled in the past.
- We also implemented a trigger to check ward capacity before admission.
- A trigger automatically adds lab test cost to billing.
- These triggers ensure consistency without relying on application logic.
- They help prevent invalid or inconsistent data entry.
- Overall, they enhance automation, integrity, and reliability of the system.

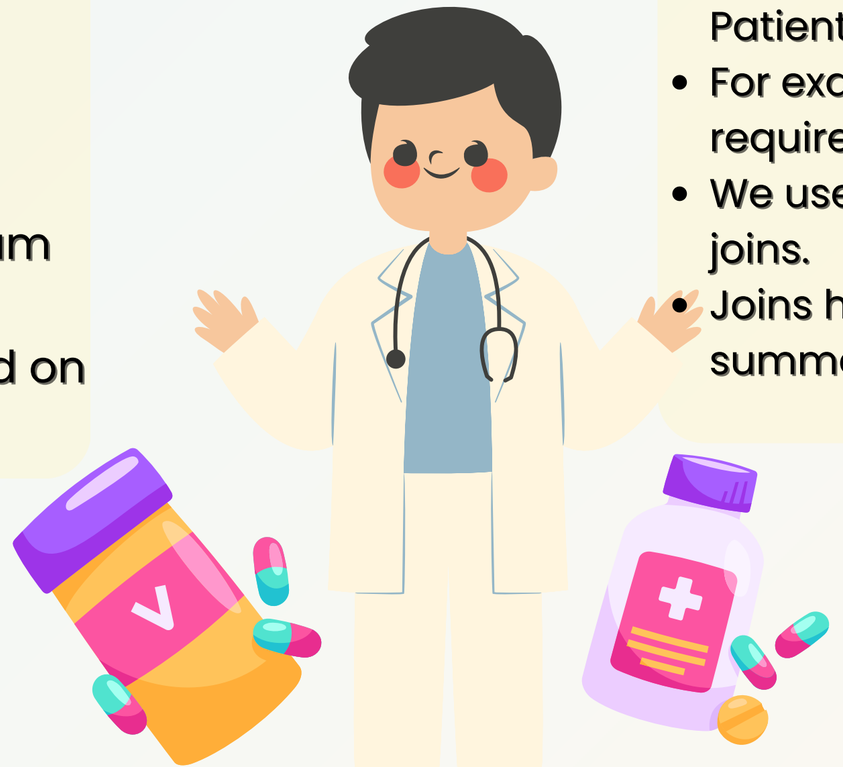
Procedures

- Stored procedures were used to implement key hospital workflows such as patient admission, billing, and appointment handling.
- They encapsulate complex SQL logic into reusable and modular units, reducing code repetition.
- Procedures improve performance because they are precompiled and executed efficiently by the database.
- They enhance security by restricting direct access to tables and allowing controlled execution of operations.
- Overall, procedures ensure consistency, maintainability, and efficient handling of real-world hospital processes.

Queries

Subqueries

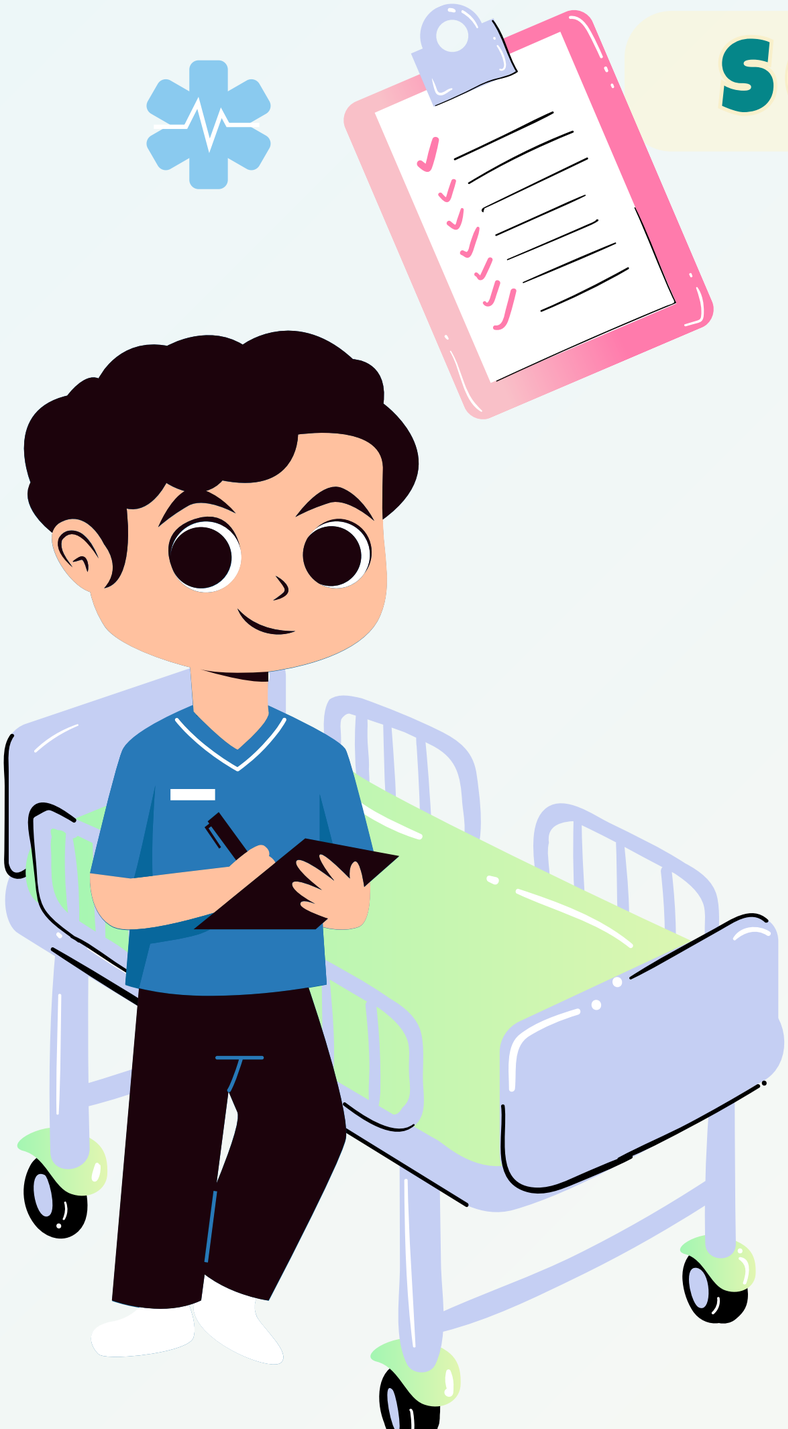
- Subqueries were used to perform nested and conditional data retrieval.
- They allow queries inside other queries for complex filtering.
- The thought process was to handle analytical queries efficiently.
- For example, finding patients with above-average billing amounts.
- Identifying doctors treating maximum number of patients.
- Subqueries help retrieve data based on aggregated results.



JOINS

- Joins were heavily used to combine data from multiple related tables.
- They are essential in relational databases to retrieve meaningful information.
- Our thought process was to connect entities like Patient, Doctor, and Treatment.
- For example, retrieving complete patient history requires multiple joins.
- We used INNER JOIN, LEFT JOIN, and multi-table joins.
- Joins help in generating reports like billing summaries and appointment details.

Security & Login System



- We implemented a role-based access control (RBAC) system.
- Roles such as Doctor, Nurse, and Admin were created.
- The thought process was to restrict access based on responsibilities.
- Doctors can access patient records and treatments.
- Nurses can view patient details and update ward-related data.
- Admin has full control over all tables and operations.
- Permissions were granted using GRANT statements on tables and views.
- Views were used to hide sensitive data like salaries.
- This ensures confidentiality and controlled data access.
- The login system maps users to roles with specific privileges.

THANK YOU

Q And A Session